

Práctica Kubernetes (MiniKube)

Elaborado por: Prof. Oscar Mondragón

1. Objetivo

Comprender el funcionamiento de Kubernetes, una plataforma de orquestación de contenedores.

2. Herramientas a utilizar

- Docker
- Kubernetes (MiniKube)
- Vagrant
- VirtualBox

3. Desarrollo de la práctica

NOTA: si tu instalación local de MiniKube presenta problemas, puedes usar uno de los siguientes playgrounds para la práctica:

Katacoda

<https://www.katacoda.com/courses/kubernetes/playground>

Play with Kubernetes

<https://labs.play-with-k8s.com/>

Tutoriales Interactivos en la página de Kubernetes

<https://kubernetes.io/docs/tutorials/kubernetes-basics/>

3.1. Vagrantfile

Esta práctica la desarrollaremos usando un Box de Ubuntu 20.04 en Vagrant **con al menos dos CPUs y 3GB de memoria**. El Vagrantfile que usaremos es el siguiente:

```
# -*- mode: ruby -*-
# vi: set ft=ruby :

Vagrant.configure("2") do |config|

  config.vm.define :servidorUbuntu do |servidorUbuntu|
    servidorUbuntu.vm.box = "bento/ubuntu-20.04"
    servidorUbuntu.vm.network :private_network, ip: "192.168.100.3"
    servidorUbuntu.vm.hostname = "servidorUbuntu"
    servidorUbuntu.vm.provider "virtualbox" do |v|
      v.cpus = 2
      v.memory = 3072
    end
  end
end
```

3.2. Instalar y Ejecutar Minikube en Linux

En esta práctica utilizaremos Minikube, una herramienta para correr Kubernetes localmente. Minikube corre un cluster Kubernetes de un único nodo en una máquina virtual.

Instala MiniKube

```
curl -Lo minikube https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64 \
&& chmod +x minikube
```

Para tener disponible en la consola el comando MiniKube, puedes añadir el comando al \$PATH o moverlo por ejemplo a /usr/local/bin:

```
sudo cp minikube /usr/local/bin && rm minikube
```

Instala kubectl, una interfaz de línea de comandos para ejecutar comandos en clusters Kubernetes.

```
sudo apt-get update && sudo apt-get install -y apt-transport-https
curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -
echo "deb https://apt.kubernetes.io/ kubernetes-xenial main" | sudo tee -a /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
sudo apt-get install -y kubectl
```

3.3. Desplegar un cluster de Kubernetes para desarrollo local usando MiniKube

Crea el grupo de Docker y agrega el usuario vagrant:

1. Crea el grupo docker (es probable que ya esté creado)

```
$ sudo groupadd docker
```

2. Agrega tu usuario al grupo Docker

```
$ sudo usermod -aG docker $USER
```

3. Cierra sesión en la máquina vagrant y vuelve a ingresar

```
$ exit  
$ vagrant ssh servidorUbuntu
```

Verifica que MiniKube está instalado correctamente, corriendo el comando para ver su versión:

```
$ minikube version
```

Inicia el clúster

```
$ minikube start  
Starting local Kubernetes cluster...  
Kubernetes is available at https://192.168.99.100:8443.  
Kubectl is now configured to use the cluster.
```

Verifica la versión del clúster

```
kubectl version
```

Visualiza detalles del cluster creado

```
kubectl cluster-info
```

Para mirar los nodos creados en el clúster, ejecuta:

```
$ kubectl get nodes
NAME      STATUS AGE
minikubevm Ready 3d
```

Inspecciona los pods que están corriendo

```
$ kubectl get pods --all-namespaces
NAMESPACE NAME                                READY STATUS RESTARTS AGE
kube-system kube-addon-manager-minikubevm 1/1 Running 3      3d
kube-system kubernetes-dashboard-06y41    1/1 Running 3      3d
```

3.3. Instalar el pod hello-minikube

MiniKube viene con una imagen Docker preconstuida llamada hello-minikube. Ejecuta la imagen hello-minikube

```
$ kubectl create deployment hello-minikube --image=gcr.io/google_containers/echoserver:1.4 --port=8080
deployment.apps/hello-minikube created
```

Ahora puedes ver los pods y deployments para verificar que han sido actualizados ejecutando los siguientes comandos:

```
$ kubectl get pods
NAME                                READY STATUS RESTARTS AGE
hello-minikube-6d4df66d87-ql9fh    1/1 Running 0      119s
```

```
~$ kubectl get deployments
NAME      READY UP-TO-DATE AVAILABLE AGE
hello-minikube 1/1 1      1      2m51s
```

Ver los eventos del clúster:

```
$ kubectl get events
```

Ver la configuración kubectl:

```
$ kubectl config view
```

Crear un service

Para acceder al servicio de hello-minikube, primero debes exponer el deployment a la red externa vía el comando:

```
$ kubectl expose deployment hello-minikube --type=NodePort
service/hello-minikube exposed
```

El flag `--type=NodePort` expone el Servicio en la IP de cada Nodo en un puerto estático (el NodePort).

Puedes verificar si el servicio fue expuesto apropiadamente listando los servicios:

```
$ kubectl get services
NAME          TYPE        CLUSTER-IP   EXTERNAL-IP  PORT(S)          AGE
hello-minikube NodePort    10.104.149.73 <none>       8080:32479/TCP   39s
kubernetes    ClusterIP   10.96.0.1     <none>       443/TCP          24h
```

Ahora puedes usar el web browser o el comando `curl` para acceder al servicio.

Para chequear la IP externa asignada y el puerto, usa el comando:

```
$ minikube service hello-minikube --url
http://172.17.0.2:32479
```

```
$ curl $(minikube service hello-minikube --url)
CLIENT VALUES:
client_address=172.18.0.1
command=GET
real path=/
query=nil
request_version=1.1
request_uri=http://172.17.0.2:8080/

SERVER VALUES:
server_version=nginx: 1.10.0 - lua: 10001

HEADERS RECEIVED:
accept=/*/*
host=172.17.0.2:32479
user-agent=curl/7.68.0
BODY:
-no body in request-
```

Nota que puedes encontrar la dirección ip usando el comando `minikube ip`

Puedes verificar los logs del pod con el comando `kubectll logs -f <nombre del pod>`. El nombre del pod lo puedes verificar con `kubectll get pods`. En los logs puedes revisar la petición get que se hizo con la prueba:

```
vagrant@servidorUbuntu:~$ kubectll logs -f hello-minikube-6d4df66d87-ql9fh
172.18.0.1 -- [07/Oct/2020:14:09:31 +0000] "GET / HTTP/1.1" 200 388 "-" "curl/7.68.0"
```

Dashboard Kubernetes

Para abrir el tablero de Kubernetes dashboard en un navegador, puedes seguir los siguientes pasos, **en dos terminales adicionales**:

1. Abre un proxy (por defecto abre un proxy en '0.0.0.0:8001')

```
$ kubectll proxy --address='0.0.0.0' --disable-filter=true
W0925 16:00:34.496367 71864 proxy.go:167] Request filter disabled, your proxy is vulnerable to XSRF attacks,
please be cautious
Starting to serve on [::]:8001
```

2. Ejecuta el dashboard **en otro terminal dentro del servidor**

```
$ minikube dashboard
🐼 Verifying dashboard health ...
🚀 Launching proxy ...
🐼 Verifying proxy health ...
🔓 Opening
http://127.0.0.1:39711/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/ in
your default browser...
👉 http://127.0.0.1:39711/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/
```

3. Reemplaza 127.0.0.1 por 192.168.100.3 y visita el dashboard en tu navegador usando

```
http://192.168.100.3:8001/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/
```

Eliminar deployment y servicios

Vuelve al terminal original. Una vez terminemos con el servicio de hello-minikube, podemos eliminar sus deployment y servicios asociados para liberar recursos. Ejecuta los siguientes comandos:

```
$ kubectl delete service,deployment hello-minikube
service "hello-minikube" deleted
deployment.apps "hello-minikube" deleted
```

```
$ kubectl get pods
No resources found in default namespace.
```

```
$ kubectl get services
NAME      TYPE      CLUSTER-IP  EXTERNAL-IP  PORT(S)  AGE
kubernetes ClusterIP  10.96.0.1    <none>       443/TCP  18h
```

3.4. Construir e instalar un nodo de servicio con Docker

En esta parte crearemos un pequeño nodo servidor en una imagen de Docker y lo correremos en Kubernetes.

Creemos proyecto llamado hello-node:

```
$ mkdir hello-node && cd hello-node && touch Dockerfile server.js
```

```
$ tree
.
├── Dockerfile
└── server.js
```

Ahora, creamos un servidor http que retorne una cadena como respuesta:

```
$ vim server.js
```

```
var http = require('http');
var requests=0;
var podname= process.env.HOSTNAME;
var startTime;
var host;
var handleRequest = function(request, response) {
  response.setHeader('Content-Type', 'text/plain');
  response.writeHead(200);
  response.write("Hello Kubernetes! | Running on: ");
  response.write(host);
```

```
response.end(" | v=1\n");
console.log("Running On:" ,host, "| Total Requests:", ++requests,"| App Uptime:", (new Date() - startTime)/1000 ,
"seconds", "| Log Time:",new Date());
}
var www = http.createServer(handleRequest);
www.listen(8080,function () {
  startTime = new Date();;
  host = process.env.HOSTNAME;
  console.log ("Kubernetes App Started At:",startTime, "| Running On: " ,host, "\n" );
});
```

Editamos el Dockerfile para declarar que esta imagen usará node 4.4, y el container correrá el servicio ejecutando el archivo server.js

```
$ vim Dockerfile
```

```
FROM node:4.4
EXPOSE 8080
COPY server.js .
CMD node server.js
```

Configuremos el environment Docker:

```
$ eval $(minikube docker-env
)
```

El comando `minikube docker-env` devuelve un conjunto de exportaciones de variables de entorno Bash para configurar el entorno local y reutilizar el demonio Docker dentro de la instancia de MiniKube.

Pasar esta salida a través de `eval` hace que bash evalúe estas exportaciones y las ponga en práctica.

Puedes revisar los comandos específicos que se ejecutarán en tu shell, omitiendo el paso de evaluación y ejecutando `minikube docker-env` directamente. Sin embargo, esto no realizará la configuración; la salida debe evaluarse para eso.

Construyamos la imagen (note el `'`. Al final del comando)

```
$ docker build -t hello-node:v1 .
Sending build context to Docker daemon 6.144 kB
Step 1 : FROM node:4.4
--> abb6383ef5fe
Step 2 : EXPOSE 8080
```



```
---> Running in a4330e4790a7
---> a1f021471d1c
Removing intermediate container a4330e4790a7
Step 3 : COPY server.js .
---> 1b394ed6fbaf
Removing intermediate container 31c5e5e505bf
Step 4 : CMD node server.js
---> Running in 6e62fbd3174f
---> ff428ba5193c
Removing intermediate container 6e62fbd3174f
Successfully built ff428ba5193c
```

\$ docker images

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-node	v1	ff428ba5193c	20 seconds ago	656.9 MB
...				

Ahora, podemos desplegar el pod hello-node en nuestro cluster local usando kubectl

\$ kubectl create deployment hello-node --image=hello-node:v1 --port=8080
deployment.apps/hello-node created

\$ kubectl get pods

NAME	READY	STATUS	RESTARTS	AGE
hello-node-6689b477fc-p6x98	1/1	Running	0	83s

\$ kubectl get deployments

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
hello-node	1/1	1	1	3m39s

Al igual que antes, debemos exponer el deployment para que use una dirección IP externa. En este caso usaremos el tipo NodePort.

\$ kubectl expose deployment hello-node --type=NodePort
service "hello-node" exposed

\$ kubectl get services

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
hello-node	NodePort	10.108.88.211	<none>	8080:30468/TCP	33s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	23h

\$ curl \$(minikube service hello-node --url)

Hello Kubernetes! | Running on: hello-node-6689b477fc-p6x98 | v=1

3.4. Explore la aplicación desplegada

Para ver los pods existentes ejecuta:

```
$ kubectl get pods
```

Para ver qué containers existen dentro del pod y qué imágenes se usaron para construir dicho pod ejecuta:

```
$ kubectl describe pods
```

En otro terminal ejecuta:

```
kubectl port-forward --address 0.0.0.0 service/hello-node 8090:8080
```

Prueba desde el browser:

<http://192.168.100.3:8090>

Deja corriendo el proxy y regresa a la ventana original. Obtén el nombre del pod y almacénalo en una variable POD_NAME

```
$ export POD_NAME=$(kubectl get pods -o go-template --template '{{range .items}}{{.metadata.name}}\n}}{{end}}')
```

Verifica el nombre del pod

```
$ echo Name of the Pod: $POD_NAME
```

Verifica los logs del contenedor

```
$ kubectl logs $POD_NAME
```

Ejecuta comandos en el contenedor,

Lista las variables de entorno en el contenedor

```
$ kubectl exec $POD_NAME -- env
```

Abre una sesión bash

```
$ kubectl exec -ti $POD_NAME -- bash
```

Una vez en la consola del contenedor, ejecuta:

```
cat server.js
```

Verifica que la aplicación está corriendo:

```
curl localhost:8080
```

Para cerrar la conexión con el contenedor, escribe `exit`.

3.5. Escalar la aplicación

Escala la aplicación a 4 réplicas:

```
$ kubectl scale deployments/hello-node --replicas=4
```

Para listar los deployments, usa `get deployments`:

```
$ kubectl get deployments
```

Chequea si el número de pods ha cambiado. Ahora hay 4 pods con IPs diferentes. Verificalo ejecutando:

```
$ kubectl get pods -o wide
```

Verifica los deployments:

```
$ kubectl describe deployments/hello-node
```

3.5. Load Balancing

Verifica que el servicio está haciendo load balancing del tráfico. Para verificar la IP y puertos expuestos, usar:

```
$ kubectl describe services/hello-node
```

Crea una nueva variable de entorno llamada `NODE_PORT` que tiene como valor Node Port

```
$ export NODE_PORT=$(kubectl get services/hello-node -o go-template='{{(index .spec.ports 0).nodePort}}')
```

```
$ echo NODE_PORT=$NODE_PORT
```

Luego ejecuta un `curl` hacia la IP o puerto expuestos. Ejecuta el siguiente comando **múltiples veces**,

```
$ curl $(minikube service hello-node --url)
```

O

```
$ curl <IP>:$NODE_PORT
```

Donde la `<IP>` puede ser verificada usando `kubectl describe pods`

Ejemplo:

```
Node:          minikube/172.17.0.3
```

En cada request un pod diferente deberá atender la petición. Esto demuestra que el load balancing está trabajando.

3.5. Scale Down

Para reducir el número de réplicas a 2 usar:

```
$ kubectl scale deployments/hello-node --replicas=2
```

Lista los deployments:

```
$ kubectl get deployments
```

El número de réplicas se redujo a 2. Lista el número de pods:

```
$ kubectl get pods -o wide
```

Esto debe confirmar que 2 pods fueron terminados.

3.5. Limpieza

Eliminemos el servicio y el deployment para hello-node y apaguemos MiniKube.

```
$ kubectl delete service,deployment hello-node
service "hello-node" deleted
deployment.apps "hello-node" deleted
```

```
$ minikube stop
👉 Stopping node "minikube" ...
🔴 Powering off "minikube" via SSH ...
🔴 1 nodes stopped.
```

4. Ejercicio

Realiza uno de los siguientes ejercicios:

- Investiga en qué consisten los Ingress Controller de Kubernetes y muéstralo con un escenario de prueba. Se sugiere reproducir la guía en:

<https://kubernetes.io/docs/tasks/access-application-cluster/ingress-minikube/>

6. Entregables y Evaluación

- Sustentación de la práctica

7. Referencias

- Kubectl <https://kubernetes.io/docs/user-guide/kubectl-overview/>
- Instalar MiniKube: <https://kubernetes.io/es/docs/tasks/tools/install-minikube/>
- Instalar kubectl: <https://kubernetes.io/es/docs/tasks/tools/install-kubectl/>
- Hello Minikube: <https://kubernetes.io/es/docs/tutorials/hello-minikube/>